

Helix Thready — Indexing Strategy

Table of Contents

1. Objectives & SLO budget

2. Query-path → index map

3. Relational index catalogue

3.1 Index rationale matrix

4. Full-text search (FTS)

5. Vector ANN indexes (pgvector)

6. The hot claim index (idempotent processing)

7. Index maintenance on partitioned tables

8. Verification & TDD

9. Gaps & open items

Rev	Date	Author	Change
1	2026-07-21	swarm (database)	Initial index catalogue: btree/GIN/trigram/FTS/partial/ANN + SLO tuning
2	2026-07-22	swarm (database, Pass 3)	Added the full per-index rationale matrix (§3.1: type, columns, predicate, query served, write-amplification, selectivity); a covering/ INCLUDE example for thread assembly; source-confirmed that the pgvector adapter's Search emits no tenant filter (§5)

Table of Contents

1. Objectives & SLO budget

2. Query-path → index map

3. Relational index catalogue

◦ 3.1 Index rationale matrix

4. Full-text search (FTS)

5. Vector ANN indexes (pgvector)

6. The hot claim index (idempotent processing)

- 7. Index maintenance on partitioned tables
- 8. Verification & TDD
- 9. Gaps & open items

1. Objectives & SLO budget

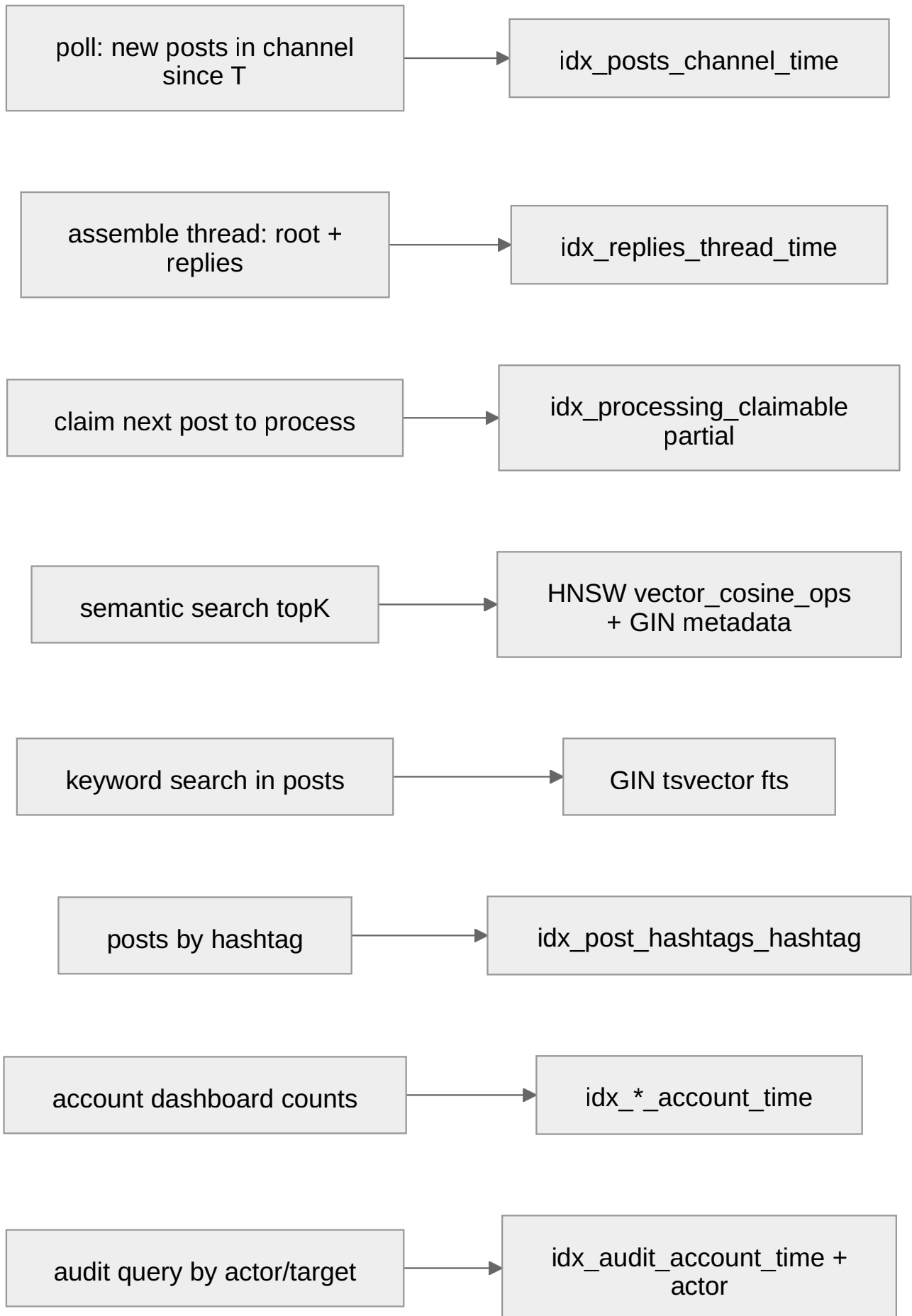
The operator chose the **Aggressive** performance tier (final request Q14): API p95 < 150 ms, semantic search < 500 ms, page < 1.5 s. Indexing must make the read paths that back those SLOs index-only or index-scan, never sequential-scan, at the Large scale (100+ channels, 10k+ posts/day, 100+ users, 50 TB+ assets — Q2). Every index below is justified by a concrete query path; we do **not** add speculative indexes (each index taxes the hot write path — 10k+ inserts/day plus replies).

Design rules:

- **Partition-prune first.** All firehose reads (`posts`, `replies`, `events`, `audit_log`) carry the time predicate so the planner prunes to a few partitions before any index scan. Children denormalise `..._posted_at` for this reason (see [erd.md §9](#)).
- **Tenant-scope every business query** with `account_id` as the leading (or included) index column.
- **Partial indexes** for skewed predicates (claimable rows, active channels).
- **GIN** for JSONB containment and array/tsvector membership; **trigram** for fuzzy text; **HNSW** for vector ANN.
- Structural PK/UNIQUE indexes live in `schema-relational.sql`; the secondary indexes below are applied by migration `0007_secondary_indexes` using `CREATE INDEX CONCURRENTLY` (see [migration-strategy.md](#)).

2. Query-path → index map

Source: `diagrams/indexing-query-paths.mmd`.



Explanation (for readers/models that cannot see the diagram). The diagram maps the nine dominant query paths (Q1 – Q9 , left) to the index that serves each (I1 – I9 , right). The point of drawing it is that every index in this document is *pulled* by a concrete query path rather than added speculatively — a discipline that matters on a firehose where each extra index taxes every insert.

The two ingestion paths anchor the design. Q1 — the messenger poller asking “what posts arrived in this channel since timestamp T” — is served by a composite btree on (channel_id, posted_at) after partition pruning, so the planner touches only the relevant month(s). Q2 — assembling a complete post (root + organic reply chain) — is served by (thread_id, posted_at) on replies, returning the chain already in reading order. Q3 — the processing engine claiming the next unit — is served by the **partial** index on processing_state(visible_at) WHERE status='pending', which stays tiny because only unclaimed rows are indexed even though the table grows one row per post forever.

The discovery and reporting paths fan out from there. Q4 — semantic search — combines the HNSW ANN index on the embedding with a GIN index on metadata for the tenant filter. Q5 — keyword/full-text search — uses a GIN index over a generated tsvector. Q6 — “all posts tagged #x” — uses the reverse index on post_hashtags(hashtag_id). Q7 — account dashboards — use (account_id, <time>) composites. Q8 — audit queries — use (account_id, created_at) plus an actor index. Q9 — asset dedup — uses the UNIQUE(account_id, content_hash) already declared on assets. The rationale matrix in §3.1 and the DDL in §3 give the exact shape of each of these indexes.

3. Relational index catalogue

Applied by 0007_secondary_indexes with CONCURRENTLY (non-blocking on live tables). On a RANGE-partitioned parent, CREATE INDEX (without CONCURRENTLY) cascades to all partitions but takes ACCESS EXCLUSIVE briefly per partition; use the per-partition CONCURRENTLY + ATTACH recipe in §7 for zero-downtime on large existing partitions.

```

-- ---- Ingestion read paths -----
-- Q1 poll: newest posts per channel (partition-pruned by posted_at)
CREATE INDEX CONCURRENTLY idx_posts_channel_time
  ON posts (channel_id, posted_at DESC);

-- Account-scoped dashboards / exports
CREATE INDEX CONCURRENTLY idx_posts_account_time
  ON posts (account_id, posted_at DESC);

-- Q2 thread assembly: replies of a thread in order
CREATE INDEX CONCURRENTLY idx_replies_thread_time
  ON replies (thread_id, posted_at ASC);

-- reply -> parent post lookups (soft ref)
CREATE INDEX CONCURRENTLY idx_replies_parent
  ON replies (parent_post_id);

-- threads by channel + recency
CREATE INDEX CONCURRENTLY idx_threads_channel_activity
  ON threads (channel_id, last_activity_at DESC);

-- active channels due for polling (partial: only active)
CREATE INDEX CONCURRENTLY idx_channels_due_poll
  ON channels (last_polled_at)
  WHERE is_active;

-- ---- Classification -----
-- Q6 posts by hashtag (reverse), and tags of a post (forward via PK)
CREATE INDEX CONCURRENTLY idx_post_hashtags_hashtag
  ON post_hashtags (hashtag_id);
CREATE INDEX CONCURRENTLY idx_reply_hashtags_hashtag
  ON reply_hashtags (hashtag_id);
-- posts of a category
CREATE INDEX CONCURRENTLY idx_post_categories_category
  ON post_categories (category_id);

-- ---- Processing -----
-- Q3 claim index is created in 0001 (see §6). Observability: failed/stuck rows.
CREATE INDEX CONCURRENTLY idx_processing_status
  ON processing_state (status, updated_at);
CREATE INDEX CONCURRENTLY idx_skill_runs_post
  ON skill_runs (post_id, skill_id);

-- ---- Assets -----
-- renditions of a raw asset
CREATE INDEX CONCURRENTLY idx_assets_parent
  ON assets (parent_asset_id) WHERE parent_asset_id IS NOT NULL;
-- links of a subject (three partial indexes keep each small)
CREATE INDEX CONCURRENTLY idx_asset_links_post
  ON asset_links (post_id) WHERE post_id IS NOT NULL;
CREATE INDEX CONCURRENTLY idx_asset_links_reply

```

```

    ON asset_links (reply_id) WHERE reply_id IS NOT NULL;
CREATE INDEX CONCURRENTLY idx_asset_links_artifact
    ON asset_links (generated_artifact_id) WHERE generated_artifact_id IS NOT NULL;
CREATE INDEX CONCURRENTLY idx_asset_links_asset
    ON asset_links (asset_id);

-- ---- Identity / RBAC hot paths -----
CREATE INDEX CONCURRENTLY idx_memberships_account
    ON memberships (account_id) WHERE status = 'active';
CREATE INDEX CONCURRENTLY idx_memberships_user
    ON memberships (user_id) WHERE status = 'active';

-- ---- Billing / metering -----
CREATE INDEX CONCURRENTLY idx_usage_account_metric
    ON usage_records (account_id, metric, window_start DESC);
CREATE INDEX CONCURRENTLY idx_usage_unbilled
    ON usage_records (account_id) WHERE NOT billed;
CREATE INDEX CONCURRENTLY idx_subscriptions_account
    ON subscriptions (account_id, status);

-- ---- Events / audit -----
CREATE INDEX CONCURRENTLY idx_events_account_time
    ON events (account_id, created_at DESC);
-- sticky last-value lookups
CREATE INDEX CONCURRENTLY idx_events_sticky
    ON events (entity_id, created_at DESC) WHERE scope = 'sticky' AND NOT invalidated;
CREATE INDEX CONCURRENTLY idx_audit_account_time
    ON audit_log (account_id, created_at DESC);
CREATE INDEX CONCURRENTLY idx_audit_actor
    ON audit_log (actor_user_id, created_at DESC);

```

3.1 Index rationale matrix

Every non-structural index, its physical shape, the exact query path it serves, and its cost. “Write-amp” is how many of these indexes a single insert into the base table must also write (the tax on the 10k+ posts/day hot path); “Selectivity” is why the index is worth that tax. Structural PK/UNIQUE indexes (declared inline in the schema) are listed at the bottom for completeness. All secondary indexes below ship in migration

`0007_secondary_indexes.sql` except the hot claim index (`0001`).

Index	Table	Type	Key / predicate	Serves	Selectivity / notes
<code>idx_posts_channel_time</code>	posts	btree	<code>(channel_id, posted_at DESC)</code>	Q1 poller: new posts in a channel since T	After partition prune, ranges one channel's newest rows; DESC matches "latest first"
<code>idx_posts_account_time</code>	posts	btree	<code>(account_id, posted_at DESC)</code>	Q7 tenant dashboards/ exports	Tenant-leading; prune + range scan
<code>idx_posts_fts</code>	posts	GIN	<code>body_fts</code> (generated tsvector)	Q5 keyword search	Complements semantic search for exact terms/code ids
<code>idx_replies_thread_time</code>	replies	btree	<code>(thread_id, posted_at ASC)</code>	Q2 thread assembly (chronological reply chain)	ASC = natural reading order; one thread per scan
<code>idx_replies_parent</code>	replies	btree	<code>(parent_post_id)</code>	reply → parent post (soft ref)	Resolves the denormalised soft FK
<code>idx_threads_channel_activity</code>	threads	btree	<code>(channel_id, last_activity_at DESC)</code>	recent active threads per channel	Powers "most recently active" lists
<code>idx_channels_due_poll</code>	channels	btree partial	<code>(last_polled_at) WHERE is_active</code>	poller: which channels are due	Partial → indexes only pollable channels; tiny
<code>idx_posts_hashtags_hashtag</code>	post_hashtags	btree	<code>(hashtag_id)</code>	Q6 "all posts tagged #x" (reverse)	Forward direction served by the <code>(post_id, hashtag_id)</code> PK
<code>idx_reply_hashtags_hashtag</code>	reply_hashtags	btree	<code>(hashtag_id)</code>	reply-tag reverse lookup	—
<code>idx_posts_categories_category</code>	post_categories	btree	<code>(category_id)</code>	posts of a content type	—
<code>idx_hashtags_trgm</code>	hashtags	GIN trigram	<code>(tag gin_trgm_ops)</code>	fuzzy/ <code>ILIKE</code> tag autocomplete	pg_trgm; short-string fuzzy match
	threads		<code>(title gin_trgm_ops)</code>		—

Index	Table	Type	Key / predicate	Serves	Selectivity / notes
<code>idx_threads_title_trigram</code>		GIN trigram		fuzzy title search	
<code>idx_processing_claimable</code>	processing_state	btree partial	<code>(visible_at) WHERE status='pending'</code>	Q3 the hot claim (<code>FOR UPDATE SKIP LOCKED</code>)	Partial → only the backlog is indexed; stays tiny forever (\$6)
<code>idx_processing_status</code>	processing_state	btree	<code>(status, updated_at)</code>	ops: stuck/failed rows, ret/backoff audit	Observability, not the hot path
<code>idx_skill_runs_post</code>	skill_runs	btree	<code>(post_id, skill_id)</code>	runs of a post / a (post,skill)	—
<code>idx_assets_parent</code>	assets	btree partial	<code>(parent_asset_id) WHERE ... IS NOT NULL</code>	renditions of a raw asset	Partial → only renditions indexed
<code>idx_asset_links_post</code>	asset_links	btree partial	<code>(post_id) WHERE ... IS NOT NULL</code>	assets attached to a post	3 partial indexes (one per subject) each stay small
<code>idx_asset_links_reply</code>	asset_links	btree partial	<code>(reply_id) WHERE ... IS NOT NULL</code>	assets attached to a reply	—
<code>idx_asset_links_artifact</code>	asset_links	btree partial	<code>(generated_artifact_id) WHERE ... IS NOT NULL</code>	assets attached to an artifact	—
<code>idx_asset_links_asset</code>	asset_links	btree	<code>(asset_id)</code>	reverse: subjects of an asset	—
<code>idx_memberships_account</code>	memberships	btree partial	<code>(account_id) WHERE status='active'</code>	RBAC: members of a tenant	Partial → active grants only; the auth hot path
<code>idx_memberships_user</code>	memberships	btree partial	<code>(user_id) WHERE status='active'</code>	RBAC: a user's tenants	—
<code>idx_usage_records</code>	usage_records	btree	<code>(account_id, metric, window_start DESC)</code>	metered-usage rollups	Billing period aggregation

Index	Table	Type	Key / predicate	Serves	Selectivity / notes
<code>unt_metric</code>				per tenant/ metric	
<code>idx_usage_unbilled</code>	usage_records	btree partial	<code>(account_id) WHERE NOT billed</code>	invoicing: open reconciliation set	Partial → shrinks as rows are billed
<code>idx_subscriptions_account</code>	subscriptions	btree	<code>(account_id, status)</code>	active subscription of a tenant	—
<code>idx_events_account_time</code>	events	btree	<code>(account_id, created_at DESC)</code>	event replay per tenant	Partitioned parent; prune + range
<code>idx_events_sticky</code>	events	btree partial	<code>(entity_id, created_at DESC) WHERE scope='sticky' AND NOT invalidated</code>	sticky last- value lookup	Partial → only live sticky events (§3.4)
<code>idx_audit_account_time</code>	audit_log	btree	<code>(account_id, created_at DESC)</code>	Q8 audit by tenant	Partitioned parent
<code>idx_audit_actor</code>	audit_log	btree	<code>(actor_user_id, created_at DESC)</code>	Q8 audit by actor	—
<code>idx_vector_hnsw_*</code> (x4)	vectordb_*	HNSW	<code>(embedding vector_cosine_ops)</code>	Q4 semantic ANN	<code>m=16, ef_construction=64 ; op</code> class matches <code><=></code> (§5)
<code>idx_vector_meta_*</code> (x4)	vectordb_*	GIN	<code>(metadata jsonb_path_ops)</code>	tenant/kind <code>@></code> filter on ANN results	Required because the adapter post-filters app-side (§5)
Structural <code>uq_users_single_root</code>	users	UNIQUE partial	<code>((is_root)) WHERE is_root</code>	enforce “exactly one root”	Guarantees ≤1 root admin (§6.1)
Structural <code>assets_account_content</code>	assets	UNIQUE	<code>(account_id, content_hash)</code>	Q9 per-tenant dedup	Content-hash idempotency

Index	Table	Type	Key / predicate	Serves	Selectivity / notes
<code>t_hash_key</code>					
Structural <code>usage_records*_key</code>	usage_records	UNIQUE	<code>(account_id, metric, window_start)</code>	idempotent metering	Upsert-on-conflict target

Write-amplification budget. `posts` carries 4 secondary indexes (`channel_time`, `account_time`, `fts`, plus the `(channel_id, external_message_id, posted_at)` UNIQUE and the PK) — every post insert writes all of them, which is the deliberate ceiling: no speculative index is added to the firehose. `replies` carries 2. `processing_state`'s only hot-path index is the tiny partial claim index. Everything heavier (GIN/trigram/HNSW) sits on lower-velocity or append-batched tables.

Covering / `INCLUDE` option for thread assembly. Q2 reads a thread's replies and usually needs `author_ref` / `raw_text` alongside the ordering. An index-only scan avoids the heap fetch:

```
-- Optional: make thread assembly index-only (Postgres 11+ INCLUDE).
CREATE INDEX CONCURRENTLY idx_replies_thread_time_cover
  ON replies (thread_id, posted_at ASC) INCLUDE (author_ref, is_system_reply);
-- Trade-off: wider index (more write cost + bloat on the firehose). Enable only if
-- EXPLAIN shows the heap fetch dominates the thread-assembly path at scale.
```

4. Full-text search (FTS)

Semantic search is the primary discovery path, but keyword FTS complements it for exact terms, code identifiers, and filtering. We add a **generated** `tsvector` column and a GIN index; `pg_trgm` supports fuzzy/`ILIKE` on short fields (titles, hashtags).

```
-- posts: generated tsvector over the message body (English config; per-lang variants TBD)
ALTER TABLE posts
  ADD COLUMN IF NOT EXISTS body_fts tsvector
  GENERATED ALWAYS AS (to_tsvector('english', coalesce(raw_text, ''))) STORED;
CREATE INDEX CONCURRENTLY idx_posts_fts ON posts USING gin (body_fts);

-- fuzzy title/tag search (trigram)
CREATE INDEX CONCURRENTLY idx_hashtags_trgm ON hashtags USING gin (tag gin_trgm_ops);
CREATE INDEX CONCURRENTLY idx_threads_title_trgm ON threads USING gin (title gin_trgm_ops);
```

[OPEN: `fts-multilang`] The generated column pins the `english` text-search config. Post content is stored in its original language (en/ru/sr-Cyrl — §12). A per-language FTS strategy (a

`lang`-driven expression index or `simple` config + language-aware ranking) is tracked as `ATM-DB-011`. Semantic search (which is language-robust via the embedding model) is the primary path meanwhile.

5. Vector ANN indexes (pgvector)

Anti-bluff (VERIFIED): `digital.vasic.vectordb`'s `pgvector` `Client.CreateCollection` creates the collection *table only* — it does **not** create an ANN index (source: `pkg/pgvector/client.go`). Without an index every `Search` is a sequential scan and the < 500 ms SLO is impossible at scale. The ANN indexes are therefore **owned by our migrations**, not the adapter. [`GAP: vectordb-3.1`]

Second VERIFIED finding (Pass 3, source-confirmed): `Client.Search` emits `SELECT id, embedding::text, metadata::text, embedding <=> $1::vector AS distance FROM <t> ORDER BY distance LIMIT $2` — with **no WHERE clause**. `SearchQuery` has a `Filter` field and `SearchQuery.Validate()` accepts it, but the adapter's SQL never references it, and only `ID + Score = 1 - distance` are returned (metadata is selected then discarded). So the ANN index alone cannot enforce tenant isolation, and the GIN `metadata` index only helps a query we issue **ourselves** (bypassing `Client.Search`) or an over-fetch we filter in Go. This is [`OPEN: vector-tenant-isolation`] / `ATM-DB-013`, and it is now confirmed at source rather than assumed — see `schema-vector.sql` header “VERIFIED CAVEAT”.

```
-- HNSW (preferred): cosine op class MUST match the query operator <=>
CREATE INDEX CONCURRENTLY idx_vec_posts_hnsw
  ON vectordb_posts USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
-- (repeat for vectordb_replies / _assets / _generated - see schema-vector.sql)
```

Tuning to hit < 500 ms :

Knob	Meaning	Start	Raise when
<code>m</code>	HNSW graph degree (build)	16	recall too low on high-dim (1024) data
<code>ef_construction</code>	build-time candidate list	64	recall too low (costs build time/memory)
<code>hnsw.ef_search</code> (session)	query candidate list	40	recall too low; higher = slower
topK over-fetch	fetch <code>topK*4</code> , then metadata-filter	×4	tenant filter under-fills topK
<code>maintenance_work_mem</code>	index build memory	2 GB	large collections build slowly

Operational guidance:

- Set `SET hns.ef_search = 40;` per search session; expose it as a config so the search service can trade latency for recall under the 500 ms budget.
- Build HNSW indexes **after** bulk backfill (faster) but keep them present during incremental writes (HNSW tolerates inserts; no retrain needed, unlike IVFFlat).
- Pair the ANN scan with `idx_vec_*_meta` (GIN on `metadata_jsonb_path_ops`) for the tenant filter; because pgvector post-filters after ANN, over-fetch topK.
- `[GAP: vectordb-3.1]` hardening path: the same `client.VectorStore` interface fronts a **Qdrant** backend with native payload filtering — swap by config if per-tenant recall or filter selectivity becomes the bottleneck. Benchmark both against the 500 ms SLO in the testing pack (`ATM-DB-012`).

6. The hot claim index (idempotent processing)

The single most latency-sensitive query is the processing engine claiming the next post. It runs continuously across the worker pool (Q4: default 32 workers) and MUST stay O(1)ish.

```
-- Created in 0001_init (see migrations/0001_init.sql):
CREATE INDEX idx_processing_claimable
ON processing_state (visible_at)
WHERE status = 'pending';

-- Claim query (one worker, exactly-once via SKIP LOCKED):
WITH claimed AS (
  SELECT post_id
  FROM   processing_state
  WHERE  status = 'pending' AND visible_at <= now()
  ORDER BY visible_at
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
UPDATE processing_state ps
SET    status = 'claimed', claimed_by = $1, claimed_at = now(), attempts = attempts + 1
FROM   claimed
WHERE  ps.post_id = claimed.post_id
RETURNING ps.post_id, ps.post_posted_at;
```

Because the index is **partial** (`WHERE status='pending'`), it only ever contains the backlog — typically small — so the ordered scan + `SKIP LOCKED` claim is cheap even though `processing_state` grows to one row per post forever. This is the in-house analogue of an exactly-once claim registry (final request §3.3, `[IN-HOUSE: background]`, `[CONSTITUTION §11.4.176]`) and closes `[GAP: session_orchestrator-2.9]` (the design-only atomic track-claim registry) for *Thready's per-post claim* by reusing the Postgres row-lock primitive rather than the unimplemented module.

7. Index maintenance on partitioned tables

- **New partitions inherit parent indexes automatically** when created with `PARTITION OF` — the maintenance job (see [partitioning.md](#)) creates partitions ahead of time so indexes exist before rows land.
- **Adding an index to an existing large partitioned table** without downtime:

```
-- 1) create the index on the PARENT as INVALID (no build), then build per partition
CREATE INDEX idx_posts_channel_time ON ONLY posts (channel_id, posted_at DESC);
-- 2) per existing partition, build CONCURRENTLY then attach
CREATE INDEX CONCURRENTLY idx_posts_channel_time_2026_07
  ON posts_2026_07 (channel_id, posted_at DESC);
ALTER INDEX idx_posts_channel_time ATTACH PARTITION idx_posts_channel_time_2026_07;
-- ... repeat per partition; parent index becomes VALID once all children attached.
```

- **REINDEX CONCURRENTLY** per partition for bloat; never on the whole parent at once.
- Keep index count lean on firehose tables — each index is written on every insert.

8. Verification & TDD

Per `[CONSTITUTION §11.4.43]` (reproduce-first) and §11.4.27 (performance/benchmark test types), each index carries an `EXPLAIN (ANALYZE, BUFFERS)` assertion in the DB test bank:

```
// RED first: assert the claim query is an Index Scan on the partial index, not Seq Scan.
func TestClaimUsesPartialIndex(t *testing.T) {
    plan := explain(t, db, claimsSQL, "worker-1")
    require.Contains(t, plan, "Index Scan using idx_processing_claimable")
    require.NotContains(t, plan, "Seq Scan")
}

// Semantic search must meet the SLO on a seeded 1e6-vector fixture.
func TestSemanticSearchP95Under500ms(t *testing.T) {
    p95 := benchSearch(t, vec, 200 /*queries*/)
    require.Less(t, p95, 500*time.Millisecond) // Q14 Aggressive SLO
}
```

These live in the testing area (`testing/`) and run against a **real** Postgres+pgvector container (mocks are unit-only per §11.4.27). Benchmarks feed the scaling/stress test types.

9. Gaps & open items

Item	Status
[GAP: <code>vectordb-3.1</code>] ANN index owned by migrations; Qdrant swap benchmarked	Addressed \$5; benchmark <code>ATM-DB-012</code>
[GAP: <code>database-3.2</code>] partitioned-index maintenance recipe	Addressed \$7
[OPEN: <code>fts-multilang</code>] per-language FTS config	Tracked <code>ATM-DB-011</code>
[OPEN: <code>vector-tenant-isolation</code>] post-filter under-fill (<code>ATM-DB-013</code>)	Addressed via over-fetch \$5; Qdrant fallback

Made with love ♥ by Helix Development.